Tokenizer Transplantation: Mitigating Autoregressive Collapse in Edge-Efficient Bengali ASR

Sanjid Hasan [1] **Md. Abdur Rahman** [2]

Abstract

Lightweight speech recognition models are critical for edge deployment, yet highly optimized architectures like Moonshine often fail on morphologically rich, non-Latin languages such as Bengali. This study identifies the root cause of this failure as the model's English-centric byte-level tokenizer, which fragments Bengali words into high-fertility byte chains and triggers catastrophic autoregressive collapse during inference. To resolve this, a novel vocabulary transplantation pipeline is proposed to replace the decoder vocabulary with the native-script BanglaBERT WordPiece vocabulary and resize the corresponding token embedding matrix. Experimental results demonstrate a reduction in token fertility from 9.16 to 1.30. By decreasing autoregressive sequence length by 85.8%, decoding instability is entirely mitigated. When evaluated on the 882-hour Lipi-Ghor dataset, the modified architecture achieves a competitive 21.54% Word Error Rate (WER) and a Real-Time Factor (RTF) of 0.0053. Ultimately, this research provides a scalable, reproducible blueprint for cross-script adaptation of compact ASR models without the need for resource-intensive pre-training.

1. Introduction

Recent advancements in Automatic Speech Recognition (ASR) have achieved remarkable performance through large-scale, self-supervised learning. However, these models often rely on broad-coverage tokenizers designed for high-resource languages, which perform suboptimally on morphologically rich languages like Bengali. This mismatch results in high token fertility, leading to increased sequence length and autoregressive collapse during inference.

In this paper, we address this bottleneck by proposing a **Tokenizer Transplantation** pipeline. Rather than training from scratch, we surgically adapt a lightweight base architecture (e.g., Moonshine) by replacing its vocabulary with a native-script BanglaBERT tokenizer. We show that by decoupling acoustic representation learning from language modeling, we can stabilize the decoder and significantly improve performance on the 882-hour Lipi-Ghor dataset. Our approach provides a scalable blueprint for adapting efficient, pre-trained architectures to under-represented languages, ensuring that the benefits of ASR reach linguistically diverse communities. The full implementation details for our framework are available in our repository.[1]

2. Related Work

2.1. Speech Representation Learning

The current ASR landscape is dominated by frameworks such as Baevski et al. (2020) and Babu et al. (2022), which leverage self-supervised learning for robust feature extraction. While models like Radford et al. (2023) have pushed the boundaries of accuracy via large-scale supervision, their computational footprint remains prohibitive for edge devices. Consequently, lightweight architectures like Moonshine (Useful Sensors, 2024; King & Sabra, 2025) have emerged as essential for local, resource-constrained deployment.

2.2. Vocabulary Adaptation

Tokenization quality is a primary determinant of model performance (Rust et al., 2021). Prior efforts, such as WECHSEL (Minixhofer et al., 2022) and FOCUS (Dobler & de Melo, 2023), have proposed innovative ways to initialize subword embeddings for cross-lingual transfer. However, these methods primarily address text-based Large Language Models (LLMs). Our work extends these principles to ASR by identifying "tokenizer transplantation" as a critical requirement for preventing autoregressive drift in morphologically rich scripts.

[1]Khulna University of Engineering & Technology, Bangladesh [2]Military Institute of Science and Technology, Bangladesh. Correspondence to: Sanjid Hasan <hasan2203090@stud.kuet.ac.bd>.

Proceedings of the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026.

[1]https://github.com/Sanjidh090/moonshine-base-bn

2.3. Bengali ASR

The development of Bengali ASR has been accelerated by datasets like Alam et al. (2022) and Faisal et al. (2023). While recent works (Tabib et al., 2026) have explored long-form ASR and speaker diarization (Bredin et al., 2020), there remains a gap in adapting state-of-the-art decoders to these languages. By integrating techniques such as TokAlign (Liu et al., 2025) and Trans-Tokenization (Delobelle et al., 2024), we provide a robust, reproducible methodology for language-specific vocabulary adaptation.

3. Tokenizer Fertility and Decoding Instability

The original Moonshine tokenizer was optimized primarily for English text. As a result, Bengali words undergo aggressive byte-level decomposition during tokenization. Tokenizer fertility is defined as the ratio of generated tokens to the total number of words:

$$\Phi = \frac{\text{Total Tokens}}{\text{Total Words}} \tag{1}$$

where lower values indicate more compact tokenization. The baseline tokenizer produces $\Phi_{baseline} = 9.16$. In contrast, the transplanted Bengali tokenizer achieves $\Phi_{transplant} = 1.30$.

The fertility gap substantially affects autoregressive decoding. Long token chains increase the probability of inference-time error accumulation (Sennrich et al., 2016). Although teacher-forced optimization remains stable, autoregressive generation becomes increasingly fragile, explaining the observed discrepancy between training loss and inference quality.

4. Methodology

To address the catastrophic decoding drift caused by high tokenizer fertility, a novel three-phase adaptation pipeline is proposed. Rather than training a model from scratch or replacing the vocabulary before fine-tuning, which often leads to catastrophic forgetting, the approach explicitly decouples acoustic representation learning from autoregressive language modeling. In this section, the sequential steps of the framework are detailed: initial acoustic adaptation of the base model, surgical transplantation of a native-script vocabulary, and a specialized recovery optimization schedule designed to stabilize the newly initialized decoder embeddings.

4.1. Phase 1: Initial Acoustic Fine-Tuning

Rather than replacing the tokenizer prior to any training which often destroys pretrained alignments and destabilizes the network our pipeline begins by fine-tuning the vanilla Moonshine-Base model on the Bengali dataset for 21 epochs. This allows the encoder to adapt its acoustic representations to Bengali phonetics. However, while the training loss converges, the decoder fails to produce coherent text due to the extreme sequence lengths generated by the native English-optimized tokenizer.

4.2. Phase 2: Tokenizer Transplantation

To resolve the decoding bottleneck while retaining the newly learned acoustic weights, tokenizer transplantation is performed on the 21-epoch fine-tuned model. The original decoder tokenizer is replaced with the BUET BanglaBERT WordPiece tokenizer (Bhattacharjee et al., 2022). Decoder embeddings are mathematically resized from the original vocabulary size (V_{old}) to the Bengali target vocabulary (V_{new}).

Special tokens are remapped explicitly to preserve decoder compatibility (e.g., mapping BanglaBERT's $\langle CLS \rangle \rightarrow BOS$ and $\langle SEP \rangle \rightarrow EOS$). The 61.5M parameter encoder architecture remains unchanged, preserving the Bengali-adapted acoustic representations learned during Phase 1.

4.3. Phase 3: Two-Stage Recovery Optimization

Because the new decoder embeddings are randomly initialized while the acoustic encoder is already fine-tuned, standard optimization leads to transplant rejection and catastrophic forgetting. To stabilize decoder adaptation, a two-stage recovery fine-tuning approach is utilized, as illustrated in Figure 1:

Stage 1 (High LR Recovery): An aggressive initial learning rate ($\eta = 2 \times 10^{-4}$) is applied using the ScheduleFree AdamW optimizer (Defazio et al., 2024). This rapidly aligns the new textual embeddings to the acoustic latent space over 7 epochs without degrading the robust encoder weights.

Stage 2 (Stabilization): Training is resumed from the 7-epoch checkpoint with a heavily decayed learning rate ($\eta = 2 \times 10^{-5}$) to fine-tune the model for grammatical and phonetic accuracy.

To maximize memory efficiency, BF16 Automatic Mixed Precision (AMP), gradient checkpointing, and gradient accumulation (step 8) are applied to achieve an effective batch size of 32 throughout both stages.

5. Experimental Setup

All model adaptations and recovery fine-tuning schedules were implemented using PyTorch, leveraging a local **NVIDIA GeForce RTX 4070** hardware accelerator with **12.9 GB VRAM** to support memory-efficient execution via Automatic Mixed Precision (AMP) and gradient checkpointing. Transcript evaluations, Word Error Rate (WER) met-

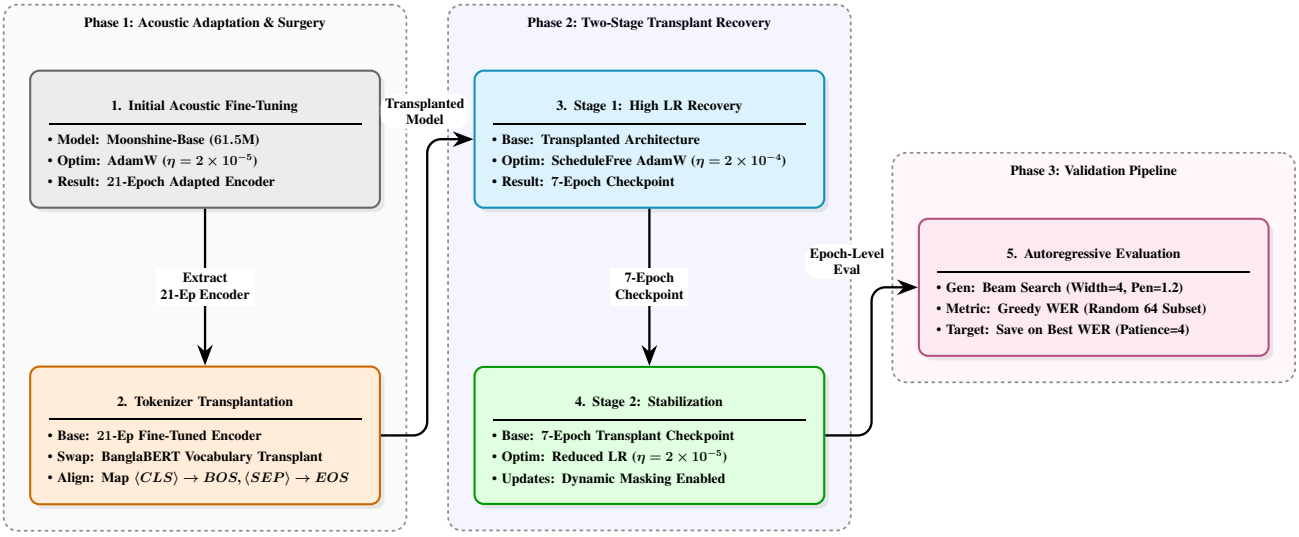

Figure 1. Overview of the Tokenizer Transplantation Methodology.

ric verification computations, and Real-Time Factor (RTF) speed benchmarking profiles were executed using the cloud-hosted **Kaggle** environment.

5.1. Dataset

Experiments utilize the Lipi-Ghor-882 dataset (Hasan et al., 2026), an 882-hour multi-speaker Bengali corpus curated from diverse open-source media. Audio is normalized to 16kHz mono. Silent segments and non-speech artifacts are aggressively filtered using Voice Activity Detection (VAD) via Pyannote (Bredin et al., 2020). This builds upon foundational Bengali speech corpora such as OOD-Speech (Faisal et al., 2023) and Bengali Common Voice (Alam et al., 2022) by offering long-form, highly diverse multi-speaker data.

5.2. Evaluation Metrics

Transcription quality is evaluated using Word Error Rate (WER) and Character Error Rate (CER), while inference efficiency is measured using Real-Time Factor (RTF). During training, Greedy WER is monitored on a randomized 64-sample validation subset at the end of each epoch, utilizing beam search (width = 4, repetition penalty = 1.2). Early stopping is triggered based on the best WER, with a patience of four epochs.

6. Results

6.1. Mitigation of Autoregressive Collapse

Tokenizer transplantation dramatically reduces Bengali sequence fragmentation (Table 1). By compressing the token representations, autoregressive decoding becomes substantially more stable, completely preventing the catastrophic drift observed in the baseline.

Table 1. Tokenizer Fertility Comparison

Tokenizer	Fertility (Φ)
Original Moonshine	9.16
Transplanted Bengali (BUET)	**1.30**

6.2. End-to-End ASR Performance

To evaluate the real-world efficacy of the tokenizer transplantation pipeline, the proposed model is compared against a spectrum of zero-shot and fine-tuned baselines on the 5% held-out Lipi-Ghor test set (comprising 5,931 utterances).

As detailed in Table 2, the transplanted Moonshine-Base architecture achieves a highly competitive Word Error Rate (WER) of 21.54%, substantially outperforming both the Zero-Shot Meta MMS 1B and the Fine-Tuned Conformer baselines.

Most notably, the proposed model achieves a Character Error Rate (CER) of 10.79%, the lowest among all tested architectures. This indicates that the native-script vocabulary empowers the decoder to construct morphologically complex Bengali words with higher sub-word accuracy than the byte-fallback approach utilized by Whisper. Furthermore, this accuracy is achieved with an order-of-magnitude reduction in parameters, matching the performance of the $\sim$769M parameter Faster Whisper Medium model with only $\sim$61.5M parameters.

Table 2. Global ASR Performance Comparison on the Lipi-Ghor Test Set. The proposed transplanted Moonshine model achieves a competitive WER and the lowest CER while operating with significantly fewer parameters than the Whisper and Conformer baselines.

Pre-training Strategy	Model Architecture	Parameters	WER (%)	CER (%)
Zero-Shot	Seamless M4T-v2	∼2.3B	66.71	45.54
Zero-Shot	OpenAI Whisper large-v3	∼1.55B	84.53	72.92
Zero-Shot	Meta MMS 1B	∼1B	43.06	21.16
Zero-Shot	Hishab TITU Conformer Large	∼120M	30.51	18.23
Fine-Tuned	Conformer Baseline (fast-conformer)	∼120M	24.67	15.56
Fine-Tuned	Faster Whisper Medium (Tustugi)	∼769M	21.28	11.18
Proposed	**Moonshine Base (Transplanted Tokenizer)**	**∼61.5M**	**21.54**	**10.79**

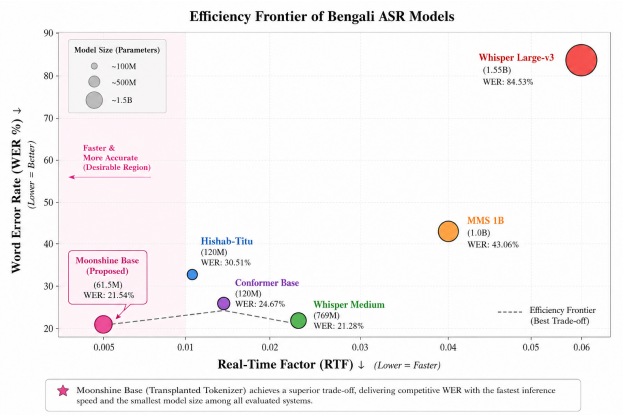

Figure 2. **Efficiency Frontier.** The proposed model (cyan) achieves a favorable trade-off between performance and efficiency.

6.3. Inference Efficiency and Edge Viability

To contextualize the computational efficiency of the proposed architecture, inference speeds were compared against top-performing baselines evaluated during the DL Sprint 4.0 Bengali ASR competition (Hasan et al., 2026). Benchmarks are calculated based on the 22-hour hidden test set utilized in the competition.

As demonstrated in Table 3, while standard architectures like Whisper-Medium required heavily engineered, parallelized CTranslate2 pipelines (Klein et al., 2017) to achieve a Real-Time Factor (RTF) of ∼0.019, the proposed Moonshine-Base with the transplanted BanglaBERT tokenizer achieves an RTF of 0.0053 natively.

Although the original vanilla Moonshine model processes the 22-hour set in slightly less time (RTF ∼0.0038), it yields catastrophic WERs near 100% (Hasan et al., 2026). Tokenizer transplantation slightly increases sequence processing time due to accurate autoregressive generation, yet successfully salvages the model's accuracy while remaining 3.5× faster than highly optimized Whisper deployments and 2.2× faster than Conformer architectures.

Table 3. Inference Efficiency on the 22-Hour DL Sprint Benchmark. The proposed model drastically outperforms competition baselines in speed while recovering accuracy.

Model Architecture	Optimization	Time (22h)	RTF
Whisper-Medium	CTranslate2 / Dual T4	∼26 min	0.0190
Hishab-Titu Conformer	Standard PyTorch	∼17 min	0.0120
Proposed Moonshine-base	**Tokenizer Transplant**	**∼7 min**	**0.0053**
Moonshine-tiny	Baseline (Failed WER)	∼5 min	0.0038

7. Conclusion

This study introduced tokenizer transplantation for the Bengali adaptation of the Moonshine ASR model. Analysis identified excessive tokenizer fertility as a primary source of autoregressive instability in compact models. By initially fine-tuning the acoustic representations, replacing the native tokenizer with a morphologically aligned Bengali tokenizer, and implementing a two-stage recovery training pipeline, inference stability on the Lipi-Ghor dataset was substantially improved. These findings demonstrate that decoupling acoustic adaptation from vocabulary alignment serves as a critical, underexplored strategy for democratizing lightweight ASR systems in low-resource languages.

Acknowledgments

The authors express their sincere gratitude to the Department of Computer Science and Engineering (CSE) at Khulna University of Engineering & Technology (KUET) for providing access to the specialized PC environment used to execute the fine-tuning and core adaptation stages of this research.

References

Alam, S. et al. Bengali common voice speech dataset for automatic speech recognition. *ResearchGate*, 2022.

Babu, A., Wang, C., Tjandra, A., et al. Xls-r: Self-supervised cross-lingual speech representation learning at scale. In *Proceedings of Interspeech*, 2022.

Baevski, A., Zhou, H., Mohamed, A., and Auli, M. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Bhattacharjee, A., Hasan, M. T., Ahmad, W., et al. Banglabert: Language model pretraining and evaluating under-resourced language nlp. In *Findings of the Association for Computational Linguistics: EMNLP*, 2022.

Bredin, H., Yin, R., Coria, J. M., et al. pyannote.audio: neural building blocks for speaker diarization. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020.

Defazio, A., Mishchenko, K., and Bottou, L. The road less scheduled. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Delobelle, P. et al. Trans-tokenization and cross-lingual vocabulary transfers: Language adaptation of llms for low-resource nlp. *arXiv preprint arXiv:2408.04303*, 2024.

Dobler, K. and de Melo, G. Focus: Effective embedding initialization for monolingual specialization of multilingual models. In *Proceedings of the Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Faisal, F. et al. Ood-speech: A large bengali speech recognition dataset for out-of-distribution benchmarking. *arXiv preprint arXiv:2305.09688*, 2023.

Hasan, S. et al. Make it hard to hear, easy to learn: Long-form bengali asr and speaker diarization. *arXiv preprint arXiv:2602.23070*, 2026.

King, J. and Sabra, A. Flavors of moonshine: Tiny specialized asr models for edge devices. *arXiv preprint arXiv:2509.02523*, 2025.

Klein, G., Kim, Y., Deng, Y., Senellart, J., and Rush, A. M. Opennmt: Open-source toolkit for neural machine translation, 2017.

Liu, Y. et al. Tokalign: Efficient vocabulary adaptation via token alignment. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2025.

Minixhofer, B. et al. Wechsel: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models. In *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.

Radford, A., Kim, J. W., Xu, T., et al. Robust speech recognition via large-scale weak supervision. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

Rust, P., Pfeiffer, J., Vulić, I., Litschko, S., and Glavaš, G. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2021.

Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2016.

Tabib, H. M. S., Rifti, I. A., Ehsan, A. M. A., Dasgupta, S., Sowdha, M. Z. M. S., Sarker, A. J., Hasan, M. M., Saha, A., Nobo, M. N. M., Bhattacharjee, S., Bhomik, T., Swapnil, A. N., and Kabir, S. Bengali-loop: Community benchmarks for long-form bangla asr and speaker diarization, 2026.

Useful Sensors. Moonshine: Lightweight speech recognition models. GitHub repository, 2024.